UNIT 11 60 min

Errors and Exceptions

Read a traceback, catch expected failures with try/except, and raise errors when input is wrong.

By the end of this unit you will be able to

- Read a traceback and identify the exception type, message, and line
- Use `try` / `except` to handle a failure you expect
- Catch a specific exception rather than everything, and explain why
- Use `else` and `finally` with a try block
- Raise an exception from your own function when input is invalid

An exception stops the program

When Python hits a problem it cannot resolve, it raises an **exception**. If nothing catches it, the program stops and prints a **traceback**. Read a traceback from the bottom up: the last line gives the exception type and message, and the lines above show the path that led to it.

traceback.py

```
def average(values):
    return sum(values) / len(values)

print(average([]))
```

The bottom line reads `ZeroDivisionError: division by zero`. The line above shows that `average` failed on `sum(values) / len(values)`, because the list was empty.

try and except

Put code that might fail inside `try`. If it raises an exception, Python jumps to the matching `except` block instead of stopping. Use this for failures you expect and can recover from, such as a user typing text where a number belongs.

safe_int.py

```
def safe_int(text):
    try:
        return int(text)
    except ValueError:
        return None

print(safe_int("42"))
print(safe_int("forty"))
```

Catch specific exceptions

Name the exception you expect. A bare `except:` or `except Exception:` also catches bugs you did not mean to handle, such as a typo in a variable name, and hides them. Catch the smallest set that makes sense, and handle each one differently when needed.

specific.py

```
def read_score(items, index):
    try:
        value = int(items[index])
    except IndexError:
        return "no such position"
    except ValueError:
        return "not a number"
    return value

print(read_score(["90", "x"], 0))
print(read_score(["90", "x"], 1))
print(read_score(["90", "x"], 5))
```

> **Watch out**
> A bare `except:` catches even `KeyboardInterrupt`, which is how a user stops a program. Avoid it. Catch named exceptions.

else and finally

- `else` runs only if the `try` block finished without an exception. Keep it for the code that depends on success.
- `finally` runs every time, whether or not an exception happened. Use it to clean up, such as closing a file.

else_finally.py

```
try:
    number = int("15")
except ValueError:
    print("bad input")
else:
    print("doubled:", number * 2)
finally:
    print("check finished")
```

Raising your own exceptions

`raise` starts an exception on purpose. It is how a function reports that the input is wrong. The caller can then catch it, or let it stop the program. Choose a standard type that matches the problem: `ValueError` for a bad value, `TypeError` for a wrong type.

raise.py

```
def set_age(age):
    if not isinstance(age, int):
        raise TypeError("age must be a whole number")
    if age < 0:
        raise ValueError("age cannot be negative")
    return age

try:
    set_age(-4)
except ValueError as error:
    print("Rejected:", error)
```

Ask forgiveness or check first

Python programmers often try the operation and handle the failure ("easier to ask forgiveness than permission"). The other style checks first with an `if`. Both are valid. Use the check when it is simple and clear, such as `if divisor != 0`. Use `try` when the check is complicated or the failure can happen in many places.

Common exceptions

Exception	Usually means	Example that raises it
`NameError`	Name not assigned	`print(total)` before `total = 0`
`TypeError`	Operation on the wrong type	`"5" + 5`
`ValueError`	Right type, bad value	`int("x")`
`IndexError`	Position out of range	`[1, 2][5]`
`KeyError`	Key not in dictionary	`{"a": 1}["b"]`
`ZeroDivisionError`	Division by zero	`1 / 0`
`FileNotFoundError`	File does not exist	`open("missing.txt")`

Predict the output THINK FIRST

Trace the flow: which block runs, and does the program keep going?

```
print(int("12") + 1)
```

Answer: `13`

```
try:
    print(1 / 0)
except ZeroDivisionError:
    print("caught")
```

Answer: `caught`

```
try:
    x = int("a")
except ValueError:
    print("bad")
else:
    print("ok")
```

Answer: `bad`
The `else` block is skipped because an exception happened.

```
try:
    print("a")
finally:
    print("done")
```

Answer: `a done`

```
try:
    raise ValueError("no")
except Exception as e:
    print(type(e).__name__, e)
```

Answer: `ValueError no`

Practice

▼ **Exercise 1 — Safe divide**

Write `safe_divide(a, b)` that returns `a / b`, or `None` when `b` is zero. Use `try` / `except`. Test it with `(10, 2)` and `(10, 0)`.

▼ **Exercise 2 — Validate a score**

Write `check_score(score)` that raises `TypeError` if the score is not an `int`, and `ValueError` if it is not between 0 and 100. Otherwise it returns the score. Call it with `105` inside a `try` block and print the error message.

▼ **Hint**

Use `isinstance(score, int)` for the type check, then a range check.

Discussion

1. Why is a bare `except:` dangerous? Describe a bug it could hide.
2. A user types "twelve" into a grade form. Should the program crash, show a message, or retry? Who decides?
3. When would you rather check with an `if` than with `try`?

Key terms

Exception

An error signal raised when something goes wrong during execution. Unhandled exceptions stop the program.

Traceback

The report Python prints for an unhandled exception, listing the call path and ending with the exception type and message.

try / except

A block that runs code and jumps to a handler if a named exception is raised.

else (try)

A block that runs only when the try block completed without an exception.

finally

A block that always runs after try/except, used for cleanup.

raise

A statement that raises an exception on purpose.

ValueError

Raised when an argument has the right type but an inappropriate value.

TypeError

Raised when an operation is applied to a value of the wrong type.

See all terms in the glossary →